

Python Operators: Bitwise vs. Boolean

This guide explains the mechanics of bitwise and boolean operators in Python, clarifying their differences and addressing specific cases like circular shifts and XOR.

1. Bitwise Operators

Bitwise operators treat operands as a sequence of bits (0s and 1s) and perform operations bit by bit.

Operator	Name	Description	Example ($5 = 101_2, 3 = 011_2$)
<code>&</code>	AND	Sets bit to 1 if both bits are 1.	$5 \& 3 = 1$ (001_2)
<code> </code>	OR	Sets bit to 1 if one of two bits is 1.	$5 3 = 7$ (111_2)
<code>^</code>	XOR	Sets bit to 1 if only one of two bits is 1.	$5 \wedge 3 = 6$ (110_2)s
<code>~</code>	NOT	Inverts all bits (Two's Complement).	$5 = -6$
<code><<</code>	Zero fill left shift	Shift left by pushing zeros from the right.	$5 \ll 1 = 10$ (1010_2)
<code>>></code>	Signed right shift	Shift right by pushing copies of the leftmost bit.	$5 \gg 1 = 2$ (10_2)

2. Boolean Operators

Boolean operators are used for logical branching and evaluate the "truthiness" of a whole expression.

Operator	Name	Description
<code>and</code>	Logical AND	Returns <code>True</code> if both statements are true.
<code>or</code>	Logical OR	Returns <code>True</code> if one of the statements is true.
<code>not</code>	Logical NOT	Reverse the result, returns <code>False</code> if the result is true.

Key Differences

1. **Level of Operation:** Bitwise operates on individual bits; Boolean operates on the truth value of the whole object.
2. **Short-circuiting:** Boolean operators (`and` , `or`) short-circuit. If the first part of an `or` is `True` , the second part is never evaluated. Bitwise operators **always** evaluate both sides.
3. **Return Type:** Bitwise operators usually return integers. Boolean operators return the object that determined the result (which behaves like a boolean).

3. Does Python have a Boolean XOR?

Python does **not** have a specific keyword for boolean XOR (like `xor`). However, you can achieve boolean XOR in two ways:

1. Using `!=` : For two boolean values, `a != b` is logically equivalent to XOR.
2. Using `bool(a) ^ bool(b)` : You can use the bitwise XOR operator on boolean types because in Python, `bool` is a subclass of `int` (`True` is 1, `False` is 0).

4. Bitwise Circular Shifts (Rotation)

Python integers have arbitrary precision, meaning they don't have a fixed "width" (like 8-bit or 32-bit). Because of this, standard shifts (`<<` and `>>`) simply discard bits that move out of range or grow the number indefinitely. To perform a **Circular Shift**, we must define a bit-width (N) and manually wrap the bits around.

Implementation

```
def left_rotate(n, d, N=8):
    """
    Performs a left circular shift (rotation) of an integer n by d bits
    within a total bit width of N.
    """
    # Ensure rotation distance is within the bit width
    d = d % N
    # 1. Shift left. % (1 << N) acts as a mask to keep only N bits.
    rotated_left_part = (n << d) % (1 << N)
    # 2. Shift right to catch the bits that "fell off" the left.
    rotated_right_part = (n >> (N - d))
    return rotated_left_part | rotated_right_part

def right_rotate(n, d, N=8):
    """
    Performs a right circular shift (rotation) of an integer n by d bits
    within a total bit width of N.
    """
    # Ensure rotation distance is within the bit width
    d = d % N
    # 1. Shift right.
    rotated_right_part = (n >> d)
    # 2. Shift left to catch the bits that "fell off" the right.
    rotated_left_part = (n << (N - d)) % (1 << N)
    return rotated_right_part | rotated_left_part
```

Why it works: The Logic Breakdown

To rotate a number n within a width N by d positions, we treat the number as two parts that swap places:

1. **The Main Shift:** When shifting left by d , the bits move toward the "higher" positions. We use `(n << d) % (1 << N)` to perform the shift and then "chop off" any bits that grew beyond the N -bit limit. The modulo operator `% (1 << N)` is mathematically equivalent to a bitwise mask of N ones (e.g., `% 256` for 8-bit).
2. **The Wrap Around:** The bits that were "chopped off" need to appear on the other side. If we shift left by d , those bits are now $N - d$ positions away from the right edge. By performing `n >> (N - d)`, we move those specific high bits down to the lowest positions.
3. **The Recombination:** Finally, we use the bitwise OR (`|`) to combine the shifted part and the wrapped part. Since the two parts occupy non-overlapping bit positions within the width N , the OR operation merges them into the final rotated result.

5. Homework Exercises

Solve the following expressions manually. Assume standard 8-bit representation for bitwise operations where applicable.

Part A: Bitwise Operations

1. What is $12 \& 7$?
2. What is $15 | 10$?
3. What is $20^1 3$?
4. Calculate 10 .
5. What is $8 \ll 2$?
6. What is $64 \gg 3$?

7. Evaluate $(5 \& 3) | 4$.
8. Evaluate $(10 \gg 1) \& 7$.
9. What is the result of $1 \wedge 1 \wedge 1$?
10. Calculate (-5) .

Part B: Boolean Operations

11. Evaluate: `True or False and False`
12. Evaluate: `not (True and False)`
13. Evaluate: `(5 > 3) and (2 == 2)`
14. What does `0 or 5` return in Python?
15. What does `10 and 20` return in Python?
16. Evaluate: `not ("apple" == "orange")`

Part C: Mixed & Advanced

17. If `a = True` and `b = True`, what is the result of `a != b` (Boolean XOR)?
18. Perform an 8-bit circular left shift on 128 (10000000_2) by 1 bit.
19. What is the value of `bool(10) ^ bool(0)`?
20. Evaluate: `(7 & 1) == 1 and (8 >> 1) == 4`